

Git Workshop Part 2

Tech GC

May 2026

1. Mental Model Review: Commits, Branches, and References
2. Merge Strategies and Team Integration
3. Rebase for Clean Feature Branches
4. Interactive Rebase: Editing Local History
5. Cherry-Pick: Moving Selected Commits
6. Git Reflog: Recovery and Repair
7. Stash, Worktrees, and Partial Staging
8. Lazygit: Visual Terminal Workflow
9. Remote Collaboration Policies
10. Fork and Pull Request Workflow
11. GitHub Actions: Automation Basics
12. Capstone Exercise: Clean Up a Messy Team Repository

Section 1

1. Mental Model Review: Commits, Branches, and References

Recap from Part 1

- **Commits:** Snapshots of repository state with hash id and parent links.
- **Branches:** Lightweight, movable pointers to specific commits.
- **HEAD:** Your current location indicator (where you are writing next).
- **Remote-tracking Branches:** Local references that mirror remote branch tips after fetch/pull (e.g., `origin/main`).

Core Key Insight

Branches do not contain commits. Branches point to commits, and commits point backward to their parents.

Visualizing the Graph

```
git log --graph --oneline --decorate --all
```

What this shows you:

- Exact history topology and branch divergence.
- Current position of `HEAD` relative to local and remote tracking branches.

Detached HEAD State

The Concept: Checking out a specific commit instead of a branch detaches HEAD from any branch name.

```
git checkout abc1234  
# HEAD now points directly to a commit
```

Warning: *New commits created here can become unreachable from normal branch names once you checkout another branch.*

Quick Recovery Options

```
# Save state immediately  
git checkout -b new-branch-name  
  
# Recover after accidentally leaving  
git reflog  
git checkout -b recovered-branch abc1234
```

Understanding References

Reference Types

Type	Example	Behavior
Branch	<code>main</code> , <code>feature/login</code>	Moves forward dynamically with new commits
Tag	<code>v1.0.0</code> , <code>release-2.0</code>	Static snapshot marker; never moves
Remote-tracking	<code>origin/main</code>	Local mirror of remote state, updated by fetch/pull
Special	<code>HEAD</code> , <code>MERGE_HEAD</code>	Internal operational pointers

Useful Reference Shortcuts

- `HEAD~1` or `HEAD^` : Direct parent of current commit
- `HEAD~2` : Grandparent of current commit
- `main~3` : Three commits back from the tip of the main branch
- `origin/main@{yesterday}` : State of the tracking branch 24 hours ago

Section 2

2. Merge Strategies and Team Integration

Integration Choices Overview

Fast-Forward Merge (--ff)

Target branch tip moves straight to source branch tip. No new commit is made.

```
Before: A---B---C (main)
          \
           D---E (feature)
After:  A---B---C---D---E (main, feature)
```

Squash Merge (--squash)

Condenses all incoming changes into a single brand-new commit on the target branch.

```
Before: A---B---C (main)
          \
           D---E---F (feature)
After:  A---B---C---G (main)
```

Decision Strategy Matrix

Strategy	Best Use Case	History Style	Extra Con
Fast-Forward	Short-lived linear branches	Seamless line	No
Three-Way	Shared/Long-lived branches	Topological	Yes
<code>--no-ff</code>	Strict Feature/PR tracking	Grouped graph	Yes
Squash	Noisy local feature commits	Compact line	Yes

Team Integration Workflows

Approach A: Linear History Preference

Keep the main line strictly straightforward. Clean chronological reading.

```
git checkout feature-branch
git rebase main
git checkout main
git merge feature-branch  # Results in a clean Fast-Forward
```

Approach B: Preserved Topology Preference

Explicitly show where work started, evolved, and was integrated.

```
git checkout main
git merge --no-ff feature-branch  # Guarantees a merge node
```

Section 3

3. Rebase for Clean Feature Branches

The Mechanics of Rebase

Rebase updates the starting parent commit of your branch, replaying your local changes sequentially on top of a new base commit.

Before Rebase:

```
A---B---C (main)
      |
      D---E (feature)
```

After 'git rebase main':

```
A---B---C (main)
      |
      D'---E' (feature)
```

Note: *D' and E' contain identical code changes to D and E, but carry distinct hash signatures and unique parent identifiers.*

Rebase Execution Steps

Step 1: Update your local tracking coordinates
`git checkout main && git pull`

Step 2: Begin replay process
`git checkout feature/my-feature`
`git rebase main`

Step 3: Handle individual files if conflicts occur, then proceed
`git add conflict_resolved.txt`
`git rebase --continue`

Rebase Interruption Controls

- `git rebase --continue` : Continue replaying commits after a conflict resolution.
- `git rebase --skip` : Drop the current conflicting commit entirely.
- `git rebase --abort` : Instantly halt and return your branch to pre-rebase status.

The Cardinal Rule of Rebase

Never rebase commits that have already been pushed to a shared public repository.

If you must update a remote branch after rebasing your own feature work, protect teammates' unseen commits:

```
git push --force-with-lease # Rejects push if remote has unseen changes
```

Section 4

4. Interactive Rebase: Editing Local History

Complete Control Over Commits

Interactive rebase (`git rebase -i`) acts as a local history editor before pushing your updates up to code review.

```
# Open interactive configuration for the last 4 commits  
git rebase -i HEAD~4
```

Command Directives Reference

Directive	Alias	Intended Outcome
<code>pick</code>	<code>p</code>	Keep the commit exactly as it is
<code>reword</code>	<code>r</code>	Keep the code changes but rewrite the log message
<code>edit</code>	<code>e</code>	Stop at this commit to modify files or split it
<code>squash</code>	<code>s</code>	Combine changes into previous commit, merging messages
<code>fixup</code>	<code>f</code>	Combine changes into previous commit, discarding this message
<code>drop</code>	<code>d</code>	Erase this specific commit completely from history

Common History Edits

Workflow 1: Squashing Noise

Turn multiple intermediate “work-in-progress” commits into a clean historical entry.

```
# Todo list modification example
pick abc1234 Add login form structure
f      def5678 Fix layout typo
f      ghi9012 Add input verification rules
```

Workflow 2: Splitting a Monolithic Commit

Break one large commit into small, atomic context pieces.

```
# 1. Mark 'edit' next to the targeted commit in the interactive menu
edit abc1234 Implement complete profile module
```

```
# 2. When Git pauses at the commit, step back one snapshot:
git reset HEAD~1
```

```
# 3. Component Stage and Commit separately
git add component_a.js && git commit -m "Profile_Part_1:_Logic"
git add component_b.js && git commit -m "Profile_Part_2:_Interface"
```

```
# 4. Resume the rebase chain
git rebase --continue
```

Section 5

5. Cherry-Pick: Moving Selected Commits

Targeting Specific Commits

Cherry-pick isolates single commits from other timelines and applies them directly on top of your current branch.

“I need this precise fix, without merging that entire unfinished branch.”

```
# Extract a singular change onto your active branch  
git cherry-pick abc1234
```

Practical Application Scenarios

- **Hotfixing:** Pulling a security fix directly from an ongoing testing branch into production.
- **Backporting:** Moving bug fixes from current development releases to legacy support branches.
- **Recovery:** Salvaging valid logic structures from a feature track that was abandoned.

Range and Automation Flags

```
# replay all commits that are ancestors of master but not of HEAD  
git cherry-pick ^HEAD master
```

Section 6

6. Git Reflog: Recovery and Repair

The Local Safety Net

Reflog tracks every single movement of `HEAD` and branch updates across your local development space.

If a commit was created locally within the past 90 days, Reflog can find and restore it.

Key Mechanics

- Reflog is entirely local; it never travels via `git push` or `git clone`.
- Entries expire after configurable time limits; common defaults are 90 days for reachable entries and 30 days for unreachable ones.

Essential Diagnostics

```
# View complete sequential history of HEAD shifts  
git reflog
```

```
# Sample Output Format:
```

```
# abc1234 HEAD@{0}: reset: moving to HEAD~2
```

```
# def5678 HEAD@{1}: commit: Add database drivers
```

Emergency Recovery Solutions

Scenario A: Accidental Hard Reset Recovery

```
# Recover from an accidental destructive command like:  
git reset --hard HEAD~3  
git reflog  
# Identify pre-reset snapshot state hash (e.g., HEAD@{1})  
git reset --hard HEAD@{1}
```

Scenario B: Restoring an Accidentally Deleted Branch

```
git reflog  
# Locate the last active commit hash belonging to the deleted branch  
git branch feature-restored HEAD@{2}
```


Section 7

7. Stash, Worktrees, and Partial Staging

Git Stash: Temporary Workspace Suspension

Save incomplete modifications without generating messy commits when context switching.

```
# Stash tracked modifications  
git stash  
  
# Stash including new, untracked workspace files  
git stash -u  
  
# Describe your stashed changes clearly  
git stash push -m "In-progress_API_refactor"  
  
# Restore the most recent stash  
git stash pop  
  
# Review all saved stash layers  
git stash list
```

Git Worktree: Multi-Branch Environments

Check out multiple branches of the same repository simultaneously in separate directories.

Review a PR or execute a hotfix without interrupting or clearing your active workspace.

```
# Mount a separate branch into a sibling directory  
git worktree add ../project-hotfix hotfix/api-bug  
  
# Safely tear down directory after finishing the fix  
git worktree remove ../project-hotfix
```

Partial Staging (Patch Mode)

Interactively break down changes within a single modified file into distinct commits.

```
git add -p # Evaluates code block hunks sequentially
```

Quick Response Map

- **y** : Stage this code hunk.
- **n** : Skip staging this code hunk.
- **s** : Split the current hunk into even smaller evaluation pieces.
- **q** : Exit immediately; preserve current staging configuration.

Section 8

8. Lazygit: Visual Terminal Workflow

Terminal Interface for Git

Lazygit is a highly responsive keyboard-driven terminal UI that maps complex graph configurations onto clear layouts.

Maintains structural clarity without hiding underlying Git CLI mechanics.

```
# Launch the interface directly in your repository folder  
lazygit
```

Primary Interface Sections

- ❶ **Status:** Shows current operational context, branch names, and upstream sync delays.
- ❷ **Files:** Lists local changes with instant staging controls.
- ❸ **Branches:** Displays local tracks, remotes, and structural tags.
- ❹ **Commits:** Visualizes the commit graph history.
- ❺ **Staging / Diff Main Panel:** Shows line-by-line diff tracking.

Navigation Basics

- `k` / `j` or Arrows : Move up and down within active panels.
- `H` / `L` or Left/Right Arrows : Cycle between different panels.
- `Space` : Toggle staging for a file, hunk, or single line.
- `c` : Open the commit message interface.
- `y` : Toggle the command output window to inspect the underlying Git commands.
- `z` : Trigger an immediate undo step for the last operation.
- `Esc` or `q` : Step back / Exit the interface.

Section 9

9. Remote Collaboration Policies

Keeping Remotes Organized

Fetch vs Pull Optimization

Fetch: Downloads remote updates without modifying local working tracks
`git fetch origin`

Pull: Combines fetch with merge or rebase, depending on configuration
`git pull`

Clean Pull: Replays your local commits cleanly on top of incoming changes
`git pull --rebase`

Tracking Verification

```
# Match local branches with remote destinations explicitly  
git push -u origin feature/auth
```

```
# Inspect tracking health and divergence distances  
git branch -vv
```

Sample Output:

```
* main          abc1234 [origin/main] Fix memory leak  
feature/auth def5678 [origin/feature/auth: ahead 1, behind 2] Update  
tokens
```

Core Safety Controls

- Avoid raw `git push --force`. Always protect your upstream target lines by using `git push --force-with-lease`.
- **Prohibited Action:** Never execute history rewrites or force-pushes on long-lived default branches (e.g., `main`, `master`, `develop`).

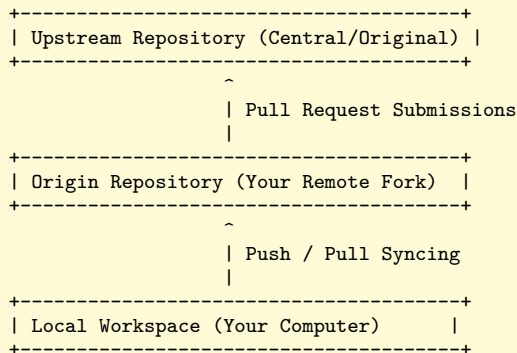
Section 10

10. Fork and Pull Request Workflow

Shared Source Management

A **Fork** is an independent, server-side copy of a repository hosted under your personal namespace.

Repository Ecosystem Map



Step-by-Step Fork Deployment

Step 1: Clone your personal fork

```
git clone [https://github.com/YOUR_USERNAME/repository.git] (https://github.com/YOUR_USERNAME/repository.git)
```

Step 2: Establish connection to the original project

```
git remote add upstream [https://github.com/ORIGINAL_OWNER/repository.git] (https://github.com/ORIGINAL_OWNER/repository.git)
```

Step 3: Synchronize with upstream changes before starting new work

```
git checkout main  
git fetch upstream  
git rebase upstream/main  
git push origin main
```

Step 4: Work on your isolated feature track

```
git checkout -b feature/contribution
```

Section 11

11. GitHub Actions: Automation Basics

Declarative Workflow Pipelines

GitHub Actions listen for repository events (e.g., pushes, pull requests) to automatically trigger builds, test suites, and deployments.

Core Concept	Functional Definition
Workflow	Automation blueprint script located within <code>.github/workflows/*.yaml</code>
Event	The explicit trigger condition (e.g., <code>push</code> , <code>pull_request</code>)
Job	A sequence of steps executed on a clean virtual machine runner
Step	Individual tasks that execute console scripts or predefined actions

Standard Continuous Integration Configuration

```
name: Project Verification Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test-suite:
    runs-on: ubuntu-latest
    steps:
      - name: Import Codebase Source
        uses: actions/checkout@v4

      - name: Initialize Runtime Environment
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Execute Standard Testing
        run: |
          npm ci
          npm test
```

Production Workflow Best Practices

- **Explicit Pinning:** Use major tags for convenience (e.g., `actions/checkout@v4`) or full commit SHAs when you need stronger supply-chain control.
- **Concurrency Controls:** Cancel older runs automatically when a developer pushes new updates to an active pull request:

```
concurrency:  
  group: ${{ github.workflow }}-${{ github.ref }}  
  cancel-in-progress: true
```

Section 12

12. Capstone Exercise: Clean Up a Messy Team Repository

The Scenario

Your team is preparing a release. The development branches are cluttered with repetitive, unverified commits, someone accidentally ran an incorrect rebase, and a vital hotfix is stuck on a abandoned branch. Your objective is to clean up this repository structure.

Practical Execution Tasks

- 1 Run the `git log --graph --oneline --decorate --all` command to map out all branch locations.
- 2 Locate and recover a lost commit using `git reflog`.
- 3 Rebase your target feature branch onto the latest `main` branch state.
- 4 Manually resolve the resulting rebase conflicts, then resume using `--continue`.
- 5 Run an interactive rebase (`git rebase -i`) to squash small commits and clean up your log messages.
- 6 Use `git cherry-pick` to bring over the isolated hotfix from the abandoned branch.
- 7 Merge the polished feature branch into `main` using your team's integration strategy.
- 8 Verify the final graph state and staging layout inside `lazygit`.
- 9 Safely push the completed history up to the remote server using `--force-with-lease`.

Thank you!